A
Master's Thesis (Capstone Project) Report
On

Autonomous Network for IMS Voice Core

Submitted by

Ashwarya Bharti

Registration Number – 24MCA10051

Guided by

Deepak SG
IMS & NextGen Voice Product Test Leader, Nokia.
(External Guide)

Dr.Anjali Mathur
Program Chair - MCA
(Internal Guide)

Submitted in partial fulfillment of the requirement for the degree of

"Master of Computer Applications"

Submitted to

School of Computing Science and Engineering

VIT Bhopal University

Bhopal (MP) – 466 114

April 2026

VIT BHOPAL UNIVERSITY, MP - 466114

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

CANDIDATE'S DECLARATION

I hereby declare that the Dissertation entitled **Autonomous Network for IMS Voice Core** is my own work conducted under the supervision of **Deepak SG** , **IMS & NextGen Voice Product Test Leader, Nokia.**

I further declare that to the best of my knowledge this report does not contain any part of work that has been submitted for the award of any degree either in this university or in other university / Deemed University without proper citation.

Ashwarya Bharti
24MCA10051

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date: 07-04-2026

Deepak SG
IMS & NextGen Voice Product Test Leader

VIT BHOPAL UNIVERSITY, MP - 466114

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the work embodied in this Capstone Project Report entitled **Autonomous Network for IMS Voice Core** has been satisfactorily completed by **Ashwarya Bharti (24MCA10051)** in the School Computing Science and Engineering at VIT Bhopal University. This report is based on project work carried out at **Nokia** under the guidance of **Mr. Deepak SG, IMS & NextGen Voice Product Test Leader, Nokia**. The work presented is a bonafide record of the candidate's effort and is submitted in partial fulfillment of the requirements for the degree of **Master of Computer Applications**.

Supervisor
Deepak SG
IMS & NextGen Voice Product Test Leader, Nokia

Forwarded by
Dr. Anjali Mathur
Program Chair - MCA

Approved by
Dr. Pushpinder Singh Patheja
Professor & Dean - SCOPE

ACKNOWLEDGEMENT

I am ineffably indebted to **Dr. G. Viswanathan (Chancellor),** Vellore Institute of Technology, Bhopal.

I would like to express my profound thanks to **Dr. Pushpinder Singh Patheja (Dean of SCOPE)** for their support and encouragement towards me at every stage in the successful completion of my project and my degree.

I would like to extend my thanks and unbound sense for the timely help and assistance given to **Dr. Anjali Mathur (Program Chair of Master of Computer Applications)** in completing the project.

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible. If there is a driving force that kept used going on in doing the project, it is the constant support of my guide **Deepak SG (IMS & NextGen Voice Product Test Leader, Nokia)**. I present my sincere and heartiest thanks to him, for giving me a patience hearing and clearing my doubts.

I take this opportunity to thank my parents and friends for their constant support and encouragement throughout this project.

EXECUTIVE SUMMARY

This report presents my experience as a Student Intern at **Nokia**, in the **Mobile Core Network Solutions (MI CNS) – Products & Engineering – IP Transport (IPT)** team, focused on **Product System Testing**.

During the internship, I was involved in two key areas: **product deployment** and **test automation**. On the deployment side, I gained hands-on experience in deploying CFX-5000, a cloud-native telecom product, in the test environment. This involved onboarding images and charts to the Harbor/Quay registry using Podman and Docker, triggering deployments using the LCM tool, and performing Helm-based (Kubernetes) deployments with security configuration.

On the automation side, I developed and enhanced **6 regression test cases** using **Robot Framework** and **Python** (paramiko, subprocess). I also wrote Python scripts for automating Secret and ConfigMap creation, and for operator-based Helm installation.

Additionally, I completed structured training in **Telecom fundamentals**, **IMS architecture**, **Kubernetes**, and **CI/CD automation**, which provided a strong technical foundation for the practical work.

This internship provided valuable exposure to real-world telecom product testing, cloud-native deployment, and test automation in a global product development environment.

List of Figures

Figure No.	Caption	Page No.
3.1	Figure 3.1 : IMS core Architecture	
3.2	Figure 3.2: Kubernetes Architecture	
4.1	Figure 4.1 : LCM-Based Deployment	
4.2	Figure 1.2: K8s Based Deployment	
5.1	Figure 2.1:Test Cases FlowChart	

List of Symbols &Abbreviations

A) Symbols

- **% :** Percentage
- **/ :** "Per" or protocol/path separator (context-based)
- **- :** Hyphen used in compound technical terms (for example, cloud-native)

B) Abbreviations ***(Alphabetical)***

- **IMS** – IP Multimedia Subsystem
- **CFX** – CFX-5000
- **CNF** – Cloud-Native Function
- **VNF** – Virtual Network Function
- **BM** – Bare Metal
- **CSCF** – Call Session Control Function
- **P-CSCF** – Proxy Call Session Control Function
- **I-CSCF** – Interrogating Call Session Control Function
- **S-CSCF** – Serving Call Session Control Function
- **HSS** – Home Subscriber Server
- **SBC** – Session Border Controller
- **SIP** – Session Initiation Protocol
- **VoLTE** – Voice over LTE
- **K8s** – Kubernetes
- **LCM** – Life Cycle Management
- **OAM** – Operations, Administration and Maintenance
- **PST** – Product System Testing
- **IPT** – IP Transport
- **MI CNS** – Mobile Core Network Solutions
- **CI/CD** – Continuous Integration / Continuous Deployment
- **UE** – User Equipment
- **TPD** – Technical Project Data

TABLE OF CONTENTS

Title Page …….. i
Declaration ……ii
Certificate …….iii
Acknowledgment ……..iv
Executive Summary……v
List of Figures……vi
List of Symbols & Abbreviations……..vii
Table of Contents …….viii

CONTENT

CHAPTER 1: INTRODUCTION……..10-14

1.1 Background……...10

1.2 Objective……...11

1.3 Proposed System……...12

1.4 Profile of the Problem and Scope of the Study……13

CHAPTER 2: ORGANIZATION AND INTERNSHIP OVERVIEW……15-16

2.1 About Nokia……..15

2.2 Department Overview – MI CNS P&E IPT……..15

2.3 Internship Role and Responsibilities……16

CHAPTER 3: TECHNICAL BACKGROUND……17-24

3.1 Telecom Fundamentals……..17

3.2 IP Multimedia Subsystem (IMS) – Architecture and Concepts……18

3.3 CFX-5000 – Product Overview and Architecture……..20

3.4 Kubernetes and Cloud-Native Technologies……..21

3.5 Automation Tools and Technologies……..23

CHAPTER 4: WORK DONE – PRODUCT DEPLOYMENT…….25-32

4.1 Deployment Overview……25

4.2 Image and Chart Onboarding to Registry……25

4.3 LCM-Based Deployment…….26

4.4 Kubernetes (Helm)-Based Deployment……...29

4.5 Post-Deployment Health Checks…….32

CHAPTER 5: WORK DONE – TEST AUTOMATION……33-36

5.1 Overview of Automation Work……33

5.2 Regression Test Cases……..33

5.3 Benefits of Automation…….36

CHAPTER 6: SYSTEM TESTING……..37-41

6.1 Testing Approach……....37

6.2 Deployment Validation Testing……..39

6.3 Regression Testing……..40

CHAPTER 7: RESULTS AND OBSERVATIONS……42-44

7.1 Deployment Results…….42

7.2 Automation Results……..43

7.3 Training Outcomes……43

CHAPTER 8: CONCLUSION……..45-46

8.1 Current State of the Work……45

8.2 Challenges Faced……..45

8.3 Future Work……..46

BIBLIOGRAPHY……47

REFERENCES…….48

APPENDICES……...…49

CHAPTER 1: INTRODUCTION

1.1 Background

The telecommunications industry has undergone a significant transformation over the past two decades, evolving from fragmented, circuit-switched voice networks toward unified, IP-based multimedia communication systems. Before the advent of the **IP Multimedia Subsystem (IMS)**, the communication world consisted of multiple independent networks – mobile, fixed-line, cable, and data – each developed independently by different standardization bodies (3GPP, ETSI, ITU-T, IEEE). There was no common concept for a unified subscriber identity, a common IP-based core for voice and multimedia services, a shared charging system, or guaranteed end-to-end Quality of Service (QoS) across different access networks.

The **IMS**, standardized by 3GPP, addresses this fragmentation by unifying all communication networks into one system. IMS provides:

- **One subscriber identity** – A single phone number or SIP URI that follows the user across home, office, and travel, regardless of device or access network.
- **One core system** – Integrating voice, data, and multimedia services into a single IP-based core network.
- **One charging system** – Common prepaid/postpaid charging independent of device, location, and access technology.
- **Seamless mobility** – Seamless handover between access technologies (e.g., WLAN to GSM/UMTS).
- **Guaranteed QoS** – End-to-end Quality of Service based on the requested service, independent of the access network.

IMS uses **SIP (Session Initiation Protocol)** as the core signaling protocol for establishing, modifying, and releasing multimedia sessions. SIP supports user location, capability negotiation, user availability, session setup, and session management – making it the foundation of all IMS call flows.

Nokia's **CFX-5000** is a cloud-native implementation of the IMS Call Session Control Function (CSCF), deployed on Kubernetes using Nokia Cloud Platform . This internship was undertaken at Nokia's **MI CNS P&E IPT (Mobile Core Network Solutions – Products & Engineering – IP Transport)** team, focused on the deployment, testing, and automation of CFX-5000 in a cloud-native test environment.

1.2 Objective

The primary objective of this internship was to gain comprehensive, hands-on experience in the deployment, testing, and automation of cloud-native telecom products at Nokia, within the **MI CNS P&E IPT** team, focused on Product System Testing.

The specific objectives are as follows:

1. Technical Training and Knowledge Building

- To develop a strong theoretical and practical understanding of telecom fundamentals, including cellular evolution, VoLTE (Voice over LTE), IMS, SIP, and Diameter/DRA protocols.
- To study the IMS architecture in depth, including its network elements, interfaces, CSCF functions (P-CSCF, I-CSCF, S-CSCF), OAM, counters, and alarms.
- To understand the different deployment variants – Bare Metal (BM), Virtual Network Function (VNF), and Cloud-Native Function (CNF).
- To develop knowledge of Kubernetes (K8s) architecture, including container management, Helm packaging, lifecycle management, storage, networking, monitoring, security, and CI/CD automation.

2. Product Deployment in the Test Environment

- To deploy CFX-5000 in the testbed using Nokia's LCM (Lifecycle Management) tool and Kubernetes-based (Helm-based) deployment methods.
- To onboard CFX-5000 images and Helm charts to the Harbor/Quay registry using Podman and Docker.
- To create and configure the necessary files for CFX-5000 deployment, including values files, Secrets, ConfigMaps, and Helm repositories.
- To perform post-deployment health checks and verify successful deployment in the test environment.

3. Test Automation

- To automate regression test cases using Robot Framework and Python (paramiko, subprocess, os), reducing manual effort and improving testing efficiency and repeatability.

- To develop Python scripts for automating the creation of Kubernetes Secrets and ConfigMaps with validation and version handling.
- To implement operator-based Helm installation commands in Python scripts.

4. Professional Development

- To develop skills in technical documentation, defect reporting, and status communication.
- To collaborate effectively with team members, mentors, and engineers across different functions within Nokia.
- To understand working processes and methodologies in a structured, global product engineering team.

1.3 Proposed System

The system studied and deployed during this internship is CFX-5000, Nokia's cloud-native implementation of the IMS CSCF, deployed in a test environment using Nokia Cloud Platform and Kubernetes.

CFX-5000 IMS Functions:

CFX-5000 provides core IMS call/session control capabilities through multiple CSCF-related functions and associated control roles.

- The **S-CSCF (Serving Call Session Control Function)** acts as the primary serving node for an IMS subscriber. It is responsible for handling user registration state, controlling SIP session signaling, and maintaining service continuity for active users. In practical terms, this function plays a central role in call/session control once the user is authenticated and attached to the IMS domain.
- The **I-CSCF (Interrogating Call Session Control Function)** works as an entry and routing control point for incoming SIP signaling. It receives incoming IMS requests, performs selection logic for the appropriate serving node, and helps route traffic to the correct internal control function. This makes it important for proper request distribution and inter-network signaling handling.
- The **E-CSCF (Emergency Call Session Control Function)** is dedicated to emergency communication handling. It applies emergency-specific routing and control behavior so

that emergency sessions are treated with higher operational priority and are directed according to emergency routing policies.

- The **BGCF (Breakout Gateway Control Function)** is used when a session needs to move from IMS/IP domain toward external telephony domains (such as PSTN interconnect scenarios). It selects the suitable breakout path/gateway, ensuring that calls are routed correctly when they need to leave the IMS service domain.
- The **EATF (Emergency Access Transfer Function)** supports emergency access transfer-related scenarios and helps maintain proper control behavior for emergency session continuity. It contributes to reliable emergency communication handling during transfer or mobility-related conditions, depending on deployment and policy configuration.

Together, these functions form an important part of IMS control behavior in CFX-5000, enabling registration control, session routing, emergency handling, and interconnect breakout management in a structured and standards-aligned manner.

1.3 Profile of the Problem and Scope of the Study

1.4.1 Profile of the Problem

The telecommunications industry is rapidly shifting from traditional network architectures to cloud-native, software-driven platforms. While this transition improves scalability and service agility, it introduces new operational and testing challenges. The major problem areas include:

1. **Complex multi-layer deployments**
   Telecom services now depend on containers, orchestration platforms, registries, and automation pipelines. A failure at any layer can impact service availability.
2. **Artifact and dependency management**
   Deployments rely on multiple images, charts, and configuration artifacts. Version mismatches or missing dependencies often lead to failed deployments.
3. **Configuration-heavy environments**
   Modern telecom platforms require large sets of parameters, credentials, certificates, and environment-specific files. Small configuration errors can cause major runtime issues.
4. **High reliability expectations**
   Telecom systems are expected to provide continuous service with minimal downtime, so deployment and testing processes must be highly accurate.

5. **Manual testing limitations**
   Manual regression validation is slow, repetitive, and difficult to scale. It can delay release cycles and increase the risk of missed defects.
6. **Troubleshooting complexity in distributed systems**
   Cloud-native telecom environments generate issues across multiple components, making root-cause analysis time-consuming without structured monitoring and automation.

1.4.2 Scope of the Study

1. **Domain Scope**

This study is limited to the testing and deployment of CFX-5000, Nokia's cloud-native IMS product, within the IP Transport domain of the Mobile Core Network Solutions team. It does not cover the full IMS product suite or other Nokia product lines outside of CFX-5000.

2. **Technology Scope**

The technologies covered within this study are limited to:

- **IMS:** Architecture, CSCF functions, SIP protocol, OAM, and call flows relevant to CFX-5000 testing.
- **Kubernetes:** Architecture, Helm, lifecycle management, storage, networking, and monitoring – only to the extent required for CFX-5000 deployment and test automation.
- **Automation tools:** Robot Framework and Python (paramiko, subprocess, os) – limited to regression test automation for CFX-5000.
- **Registry and deployment tools:** Harbor/Quay registry, Podman, Docker, Pycharm and Nokia's LCM tool.

3. **Environment Scope**

All work is carried out exclusively within the testbed. Production network environments are outside the scope of this study. The study is limited to the NCP cluster available within the Nokia IPT lab.

4. **Automation Scope**

The automation work is scoped to 6 regression test cases directly relevant to CFX-5000 deployment and operational validation. Development of a full automation framework or end-to-end test suite is beyond the scope of this study.

CHAPTER 2: ORGANIZATION AND INTERNSHIP OVERVIE

2.1 About Nokia

Nokia is a global B2B technology leader in telecommunications and network infrastructure. The company focuses on building and enabling high-performance, secure, and scalable communication networks for service providers, enterprises, and governments. Nokia's purpose is to create technology that helps the world act together, with strong emphasis on network innovation, cloud-native evolution, and digital transformation.

Nokia plays a major role in mobile and fixed network technologies, including 4G/5G, IP networking, core networks, cloud, and software platforms. In addition to network products, Nokia contributes to standards, research, and long-term innovation through continuous investment in advanced telecom technologies.

For this internship, Nokia provided a practical industrial environment where cloud-native telecom products are deployed, tested, and improved using real engineering workflows and tools.

2.2 Department Overview – MI CNS P&E IPT

The internship was carried out in the MI CNS P&E IPT organization, specifically in the CFX PST (Product System Testing) team.

This department is responsible for engineering and testing activities related to mobile core and IP transport product areas. The team works on deployment, configuration, validation, and troubleshooting of telecom products in cloud-native environments.

As part of the CFX PST System Testing team, the work focused on deployment validation, post-deployment checks, issue troubleshooting, and regression test automation to ensure product quality and operational readiness.

The department follows structured product engineering workflows and collaborates with multiple stakeholders such as development teams, integration teams, and support teams. A key focus area is ensuring product quality and operational readiness through systematic deployment and test validation in controlled testbeds.

2.3 Internship Role and Responsibilities

During the internship, I worked as a Student Intern in the CFX PST (Product System Testing) function. My responsibilities can be classified into the following categories:

A. Deployment Activities

- Assisted in product deployment in the testbed using LCM and Kubernetes/Helm workflows.
- Onboarded product images and charts to the registry for deployment.
- Prepared and updated deployment-related configuration files

B. Validation and Verification

- Supported deployment validation through health checks and status verification.
- Verified pod/service readiness and post-deployment stability in the test environment.

C. Troubleshooting and Issue Resolution

- Identified and analyzed deployment issues, including registry and configuration mismatches.
- Supported troubleshooting and redeployment activities to achieve stable deployment.

D. Test Automation

- Automated regression test scenarios using Robot Framework and Python.
- Developed Python scripts for operational automation tasks, including Secret/ConfigMap creation and Helm command automation.

E. Documentation and Reporting

- Documented deployment steps, observed issues, applied fixes, and key learnings.
- Maintained structured records for deployment and testing cycles.

CHAPTER 3: TECHNICAL BACKGROUND

This chapter explains the technical knowledge and training that supported the internship work. The internship involved cloud-native deployment, system testing, troubleshooting, and regression automation of CFX-5000 in a telecom testbed. To perform this work effectively, training was completed in telecom fundamentals, IMS architecture and operations, Kubernetes, and automation tools.

3.1 Telecom Fundamentals

The initial training covered the basic telecom concepts required to understand how call and signaling systems work in modern networks.

- **Introduction to telecom systems:**

  Learned how telecom networks are organized and how services like voice, messaging, and data are delivered over network infrastructure.

- **Telecom basics:**

  Understood common network elements, service flow, signaling path, and how subscriber requests are processed in a telecom environment.

- **Cellular evolution:**

  Studied the transition from legacy mobile systems to modern packet-based networks, and why cloud-native deployment models are now important.

- **VoLTE/IMS introduction:**

  Learned how voice is delivered over LTE using IMS, and how IMS enables multimedia communication using IP protocols instead of traditional circuit-switched methods.

- **Diameter/DRA introduction:**

  Understood how Diameter protocol is used for subscriber/session policy communication and how DRA routes and controls Diameter signaling traffic.

- **SIP tutorial:**

  Learned the role of SIP in session setup, modification, and release, and how SIP message flow is used in call establishment.

3.2 IP Multimedia Subsystem (IMS) – Architecture and Concepts

The **IP Multimedia Subsystem (IMS)** is a 3GPP-standardized framework used to provide IP-based communication services such as voice, video, messaging, and multimedia sessions. IMS is the core enabler for services like **VoLTE**, **VoWiFi**, and **5G voice (VoNR)**.

IMS was introduced to solve the limitations of legacy telecom systems, where services were distributed across separate and non-uniform networks. It provides a unified architecture in which signaling, session control, subscriber management, and service execution are integrated in a standardized way. This allows operators to deliver consistent services across different access technologies and devices.

3.2.1 IMS Architecture Overview

IMS is commonly viewed in three logical layers:

1. **Access Layer**

   Connects user equipment (UE) to the IMS core through LTE/5G/Wi-Fi/fixed broadband networks.

2. **Control Layer**

   Handles SIP signaling, registration, authentication, session routing, and policy decisions.

3. **Service/Application Layer**

   Hosts telephony and multimedia service logic through application servers and media functions.

3.2.2 Major IMS Components and Their Functions

- **P-CSCF (Proxy CSCF):** First IMS node contacted by UE; forwards SIP signaling and applies access-side policy/security.
- **I-CSCF (Interrogating CSCF):** Entry point for incoming SIP traffic; identifies the suitable serving node.
- **S-CSCF (Serving CSCF):** Main control function for registration and session management.
- **HSS (Home Subscriber Server):** Stores subscriber profile, authentication, and service data.

- **SBC (Session Border Controller):** Protects network boundaries and manages interconnect signaling/media.
- **MTAS (Multimedia Telephony Application Server):** Provides service features (e.g., supplementary telephony services).
- **MRF (Media Resource Function):** Handles media resources (announcements, conferencing, media processing).
- **BGF (Border Gateway Function):** Controls media path and RTP flow at network boundaries.

3.2.3 Cloud-Native IMS Context

Modern IMS is increasingly deployed as **VNF/CNF** on virtualized and Kubernetes environments. This cloud-native model provides:

- Better scalability (horizontal scaling of functions)
- Automation through orchestration tools
- Faster rollout and easier lifecycle management
- Improved resilience through self-healing platform behavior

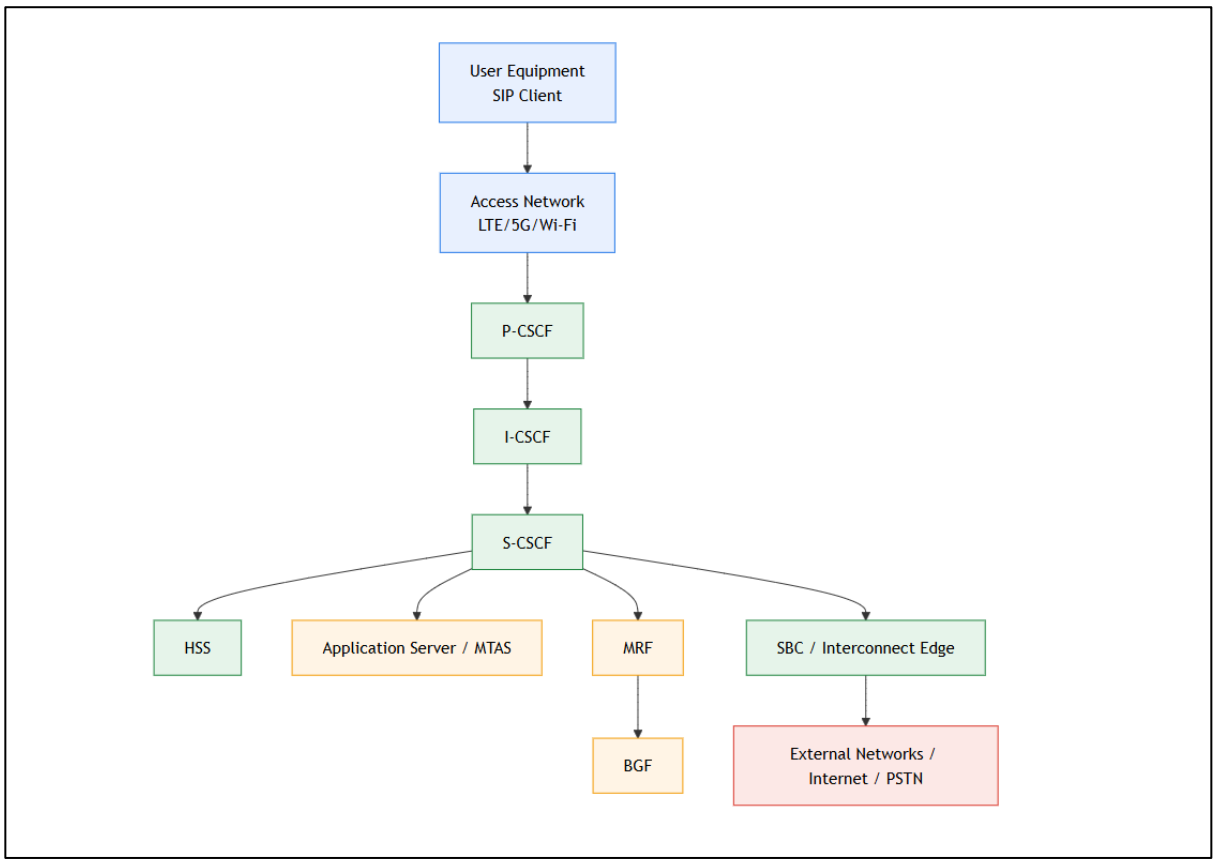

Figure 3.1 : IMS core Architecture

3.3 CFX-5000 – Product Overview

CFX-5000 was the central product in this internship and all practical deployment/testing work was performed around it.

- **Product role in IMS:**

  CFX-5000 works in the IMS call/session control domain and supports core service control functions.

- **Cloud-native deployment model:**

  In this internship context, CFX-5000 was handled as a CNF in a Kubernetes-based testbed.

- **Configuration-driven behavior:**

  Deployment depended on correct values files, onboarding artifacts, and runtime configuration consistency.

- **Operational dependency on platform components:**

Successful bring-up required correct integration with platform services, registry artifacts, and deployment tooling.

- **Deployment and post-deployment validation focus:**

  Practical work included install/upgrade activities, health verification, troubleshooting, and stabilization.

3.4 Kubernetes and Cloud-Native Technologies

Kubernetes and Helm formed the technical base for product deployment and runtime operations in the testbed. The following topics explain the key concepts used during deployment and validation work.

1. Kubernetes Introduction and Architecture

Kubernetes is a container orchestration platform that runs applications as pods across a cluster of nodes.

Its architecture separates responsibilities between control components (which decide what should run) and worker components (which actually run the containers). This model enables consistent deployment, scaling, and recovery of cloud-native applications.

2. Container Platform Tools

Container tools are used to prepare, package, and manage deployment artifacts such as images and charts.

These tools support artifact onboarding to registries and help maintain versioned, repeatable deployment workflows.

3. Lifecycle Management

Cloud-native applications follow a lifecycle: deployment, validation, monitoring, restart/recovery, scale operations, and updates.

Lifecycle management ensures that the product remains operational after installation and can be safely updated or recovered when issues occur.

4. Core Kubernetes Concepts

Key Kubernetes objects (such as pods and related deployment resources) define how an application is executed in the cluster.

Using these objects correctly is essential to run CNF workloads in a controlled test namespace and maintain stable runtime behavior.

5. Storage and Networking

Persistent storage and network configuration are critical for reliable telecom deployment.

Storage ensures required data remains available, while networking enables communication between pods, services, and external interfaces used by telecom signaling and media paths.

6. Scheduling and Runtime Behavior

Kubernetes scheduler decides where workloads run based on resource availability and policy.

Proper scheduling affects service performance and availability, especially when workloads are scaled or when node conditions change.

7. Helm Packaging

Helm provides chart-based deployment for Kubernetes applications.

A chart defines all required resources, and values files customize installation parameters. This allows repeatable and environment-specific deployments with minimal manual changes.

8. Application Healing and Health Checks

Kubernetes continuously checks container/pod health and restarts failed workloads when required.

Readiness and health checks are used after deployment to verify whether services are operational and stable.

9. Logging and Monitoring

Logs and runtime status are primary sources for identifying failures and validating behavior.

Monitoring helps detect issues early, while log analysis supports root-cause troubleshooting during deployment and testing cycles.

10. Security Basics

Cloud-native deployments require secure handling of credentials, certificates, and access boundaries.

Security controls in the environment protect deployment workflows and reduce risks related to unauthorized access or misconfiguration.

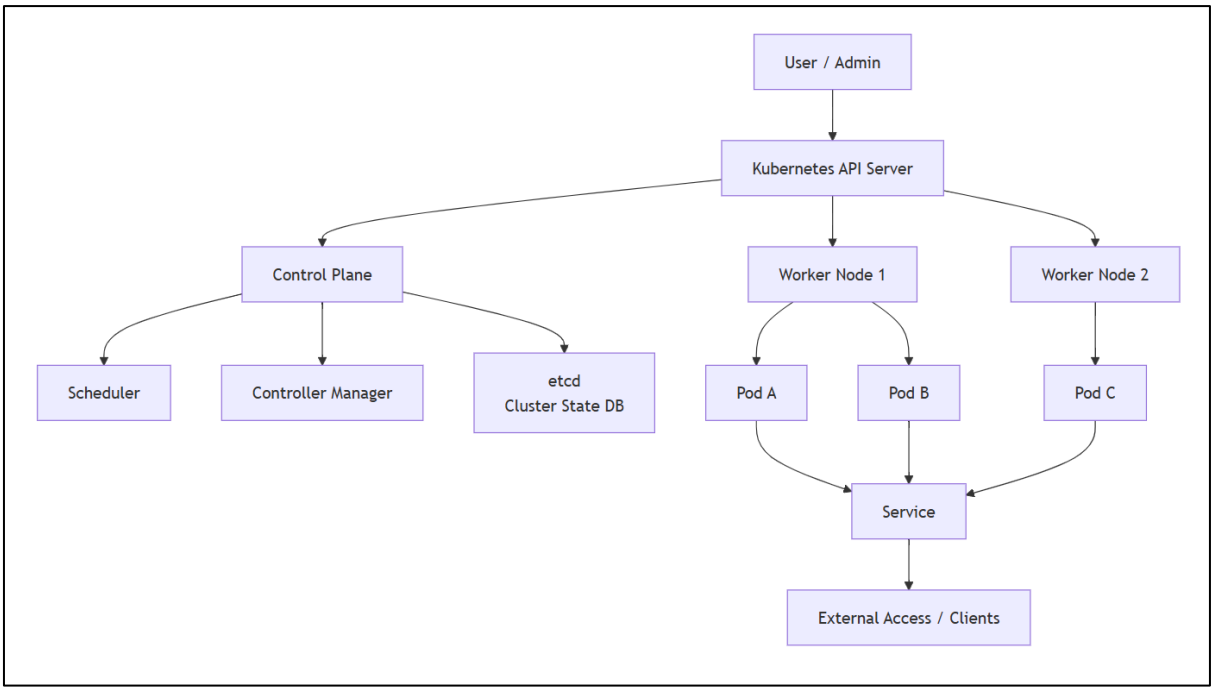

Figure 3.2: Kubernetes Architecture

3.5 Automation Tools and Technologies

Automation in this project was used to make regression validation faster, repeatable, and less error-prone. The automation stack combined Robot Framework for test execution and Python scripts for operational tasks.

1) Automation Framework and Scripting Stack

1. **Robot Framework (ROBOT)**
   - **Purpose:** Keyword-driven test automation framework for regression scenarios.
   - **Usage in project:** Execution of repeatable regression flows, restart validations, and environment checks.
   - **Version used:** Robot Framework 7.4.1
2. **Python**
   - **Purpose:** Scripting language for deployment-support automation and utility scripts.
   - **Usage in project:** Secret/ConfigMap handling, command automation, and deployment helper scripts.
   - **Version used:** Python 3.13.5

2) Python Libraries Used

1. **subprocess**
    - **Purpose:** Run shell/CLI commands from Python scripts.
    - **Usage in project:** Triggering deployment commands, Kubernetes/Helm command execution, and collecting command output for validation.
    - **Version:** Standard library 3.13.5
2. **paramiko**
    - **Purpose:** SSH-based remote execution and secure server access from scripts.
    - **Usage in project:** Running remote operations on deployment/test servers and automating command execution over SSH.
    - **Version used:** paramiko 4.0.0
3. **OS**
    - **Purpose:** File system and environment operations.
    - **Usage in project:** Path handling, file checks, directory operations, and environment-variable-based script behavior.
    - **Version:** Standard library 3.13.5
4. **wget**
    - **Purpose:** Download files/artifacts from URLs.
    - **Usage in project:** Fetching required deployment artifacts, packages, or references before onboarding/execution.
    - **Version used:** wget 1.21.4
5. **re**
    - **Purpose:** Regular expression processing for pattern matching/parsing.
    - **Usage in project:** Parsing logs, validating command output patterns, and extracting structured values from text.
    - **Version:** Standard library 3.13.5

3) CI/CD-Oriented Automation Relevance

- Standardized execution sequence reduced variation between test runs.
- Scripted operations improved repeatability across deployment cycles.
- Consistent checks made failure reproduction and troubleshooting easier.
- Automation reduced manual effort for recurring regression activities.

CHAPTER 4: WORK DONE – PRODUCT DEPLOYMENT

This chapter presents the practical deployment work completed during the internship for the cloud-native CFX-5000 setup in the test environment. The work followed the standard deployment approach described in the deployment guide: prerequisites, image/chart onboarding, configuration preparation, deployment trigger through the lifecycle management platform, and post-deployment verification.

The deployment activities were performed iteratively across multiple cycles, which helped in improving stability, reducing manual errors, and developing a repeatable deployment workflow.

4.1 Deployment Overview

The deployment work focused on bringing up **CFX-5000 CNF** in a controlled testbed environment using Nokia's cloud-native deployment workflow. The process combined registry onboarding, configuration preparation, lifecycle operations through LCM, and post-deployment validation.

The execution was iterative and done across multiple deployment cycles. Each cycle improved environment readiness, configuration accuracy, and deployment stability. The deployment activities included both one-time platform setup tasks and repeatable deployment tasks.

High-level deployment phases:

1. Pre-deployment prerequisites and environment preparation
2. Image/chart onboarding to registry
3. Configuration artifact preparation (values/inputs)
4. Triggered deployment through LCM tool
5. Post-deployment verification and issue resolution

4.2 Image and Chart Onboarding to Registry

A major part of deployment readiness was onboarding container images and Helm charts to the target registry (Harbor/Quay), as referenced in the deployment guide section for onboarding images/charts.

What was done

- Required product artifacts were collected on the deployment server.
- Images and charts were prepared in the expected format.
- Artifacts were pushed to the registry using container tools.
- Registry paths, tags, and names were verified before deployment.

Why this step is critical

- Deployment pulls artifacts only from configured registry paths.
- If image/chart names do not match deployment values, deployment fails.
- Incomplete onboarding causes partial or failed component bring-up.

Practical observations

- Registry naming consistency is essential.
- Re-upload/duplicate artifact scenarios must be handled carefully.
- Validation after onboarding reduces downstream deployment failures.

4.3 LCM-Based Deployment

After completing image/chart onboarding and configuration preparation, the product deployment was executed through the **Lifecycle Management (LCM)** workflow. This stage converted all prepared deployment artifacts into an actual running instance in the testbed and served as the primary controlled path for product bring-up.

LCM-based deployment was used because it provides a structured and auditable execution model. Instead of manually running disconnected deployment commands, the lifecycle platform organizes package upload, instance creation, deployment triggering, and status monitoring into a governed process. This improves consistency, traceability, and repeatability across deployment cycles.

Execution Flow:

1. One-time environment readiness and integration check

Before deployment execution, platform-level integration and environment readiness were validated. This included confirming that the cluster was reachable through the lifecycle

platform and that required prerequisites had been completed. This one-time validation step ensured that deployment events and package operations could be executed without platform-side blocking issues.

2. Service package upload

The network service package was uploaded to the lifecycle platform repository/catalog. This package contains the deployment definition, including the deployment order and references needed for instance creation. A successful package upload was treated as a gating step before proceeding further.

3. Instance creation using prepared configuration

After upload, a deployment instance was created using prepared input/configuration files (for example, values and instance input definitions). These files provided environment-specific parameters such as artifact references, deployment values, and operational settings required for runtime initialization.

4. Creation of dependencies

Where required, Kubernetes dependencies such as **Secrets** and **ConfigMaps** were created prior to deployment trigger. These resources were essential for secure credential handling and configuration injection. Missing or incorrect dependency resources could cause deployment to reach a partial state without becoming fully ready.

5. Deployment trigger through lifecycle workflow

Once package, instance, and dependencies were in place, deployment was triggered from the lifecycle platform. At this point, orchestration began using the uploaded package definition and supplied parameters, and the platform initiated the product bring-up sequence in the target namespace/environment.

6. Continuous monitoring of deployment state and events

Deployment progress was monitored through platform status, event streams, and runtime verification checks. Monitoring continued until a successful and stable completion state was reached. This monitoring step was important for early detection of configuration mismatches, artifact reference errors, or delayed component readiness.

FlowChart : LCM-Based Deployment

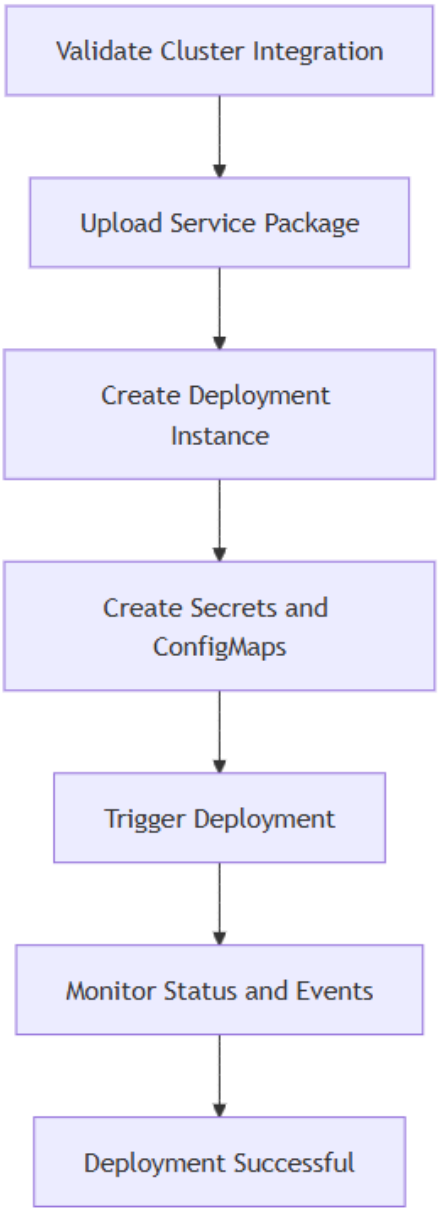

Figure 4.1:LCM Deployment

Operational Scope of LCM Workflow

The same lifecycle mechanism supports the complete operational lifecycle, not only first-time deployment. During activities, this model was recognized as the standard control path for:

- **Deployment** – Initial bring-up of product instance
- **Upgrade** – Controlled version transition
- **Rollback** – Reverting to previous stable state if needed
- **Termination** – Controlled instance removal

4.4 Kubernetes (Helm)-Based Deployment

In addition to lifecycle-driven deployment operations, a significant part of product bring-up was executed through a Kubernetes + Helm-based workflow. This approach was used for setup tasks, chart installation, profile-level configuration, and runtime verification at cluster level. While lifecycle orchestration provided process control, Helm-based deployment provided direct operational control over chart execution and release behavior in the Kubernetes environment.

1. **Deployment execution environment**
    - The Kubernetes-based deployment was executed from the deploy server using OpenShift CLI (oc) and Helm (helm).
    - The target namespace was used as the deployment scope for all resources and releases.
2. **Pre-deployment cleanup and namespace reset**
    - Old role, service account, and objects from previous runs were deleted.
    - Application profile secrets were removed.
    - Existing workloads were checked and validated as empty before fresh deployment.
    - Old ConfigMap and secrets entries that could cause conflicts were deleted.
    - Failed profile resources were cleaned by removing finalizers and confirming no profile objects remained.
3. **Secret and ConfigMap preparation**

- Required runtime secrets were recreated in the target namespace.
- Deployment-support ConfigMap was created from required input artifact(s).
- Resource creation was validated before chart installation.

4. **Helm repository setup**
    - Helm repository configuration file was prepared.
    - Helm repository chart was installed successfully in the namespace.
    - Release status was monitored to confirm successful repository setup.
5. **Operator/service chart installation**
    - Application API/operator chart was installed using Helm with values-file input.
    - Secure registry access parameters were provided during chart pull/install.
    - Helm release reached deployed state.
    - Pod readiness was monitored until full ready condition was achieved.
6. **Main product profile deployment**
    - Main profile chart was installed using:
        - Product values input
        - Service/reference values inputs
        - Additional configuration file inputs
    - Helm deployment completed with successful release status.
7. **Post-install runtime verification**
    - Release-level verification performed using Helm release listing.
    - Pod-level verification performed using namespace pod status checks.
    - Resource-level checks were done to ensure deployment artifacts were correctly created.
8. **Operational result**
    - Deployment completed successfully through a structured sequence.
    - Cleanup + dependency recreation + staged Helm installation significantly reduced deployment conflicts.

Flowchart: K8s Based Deployment

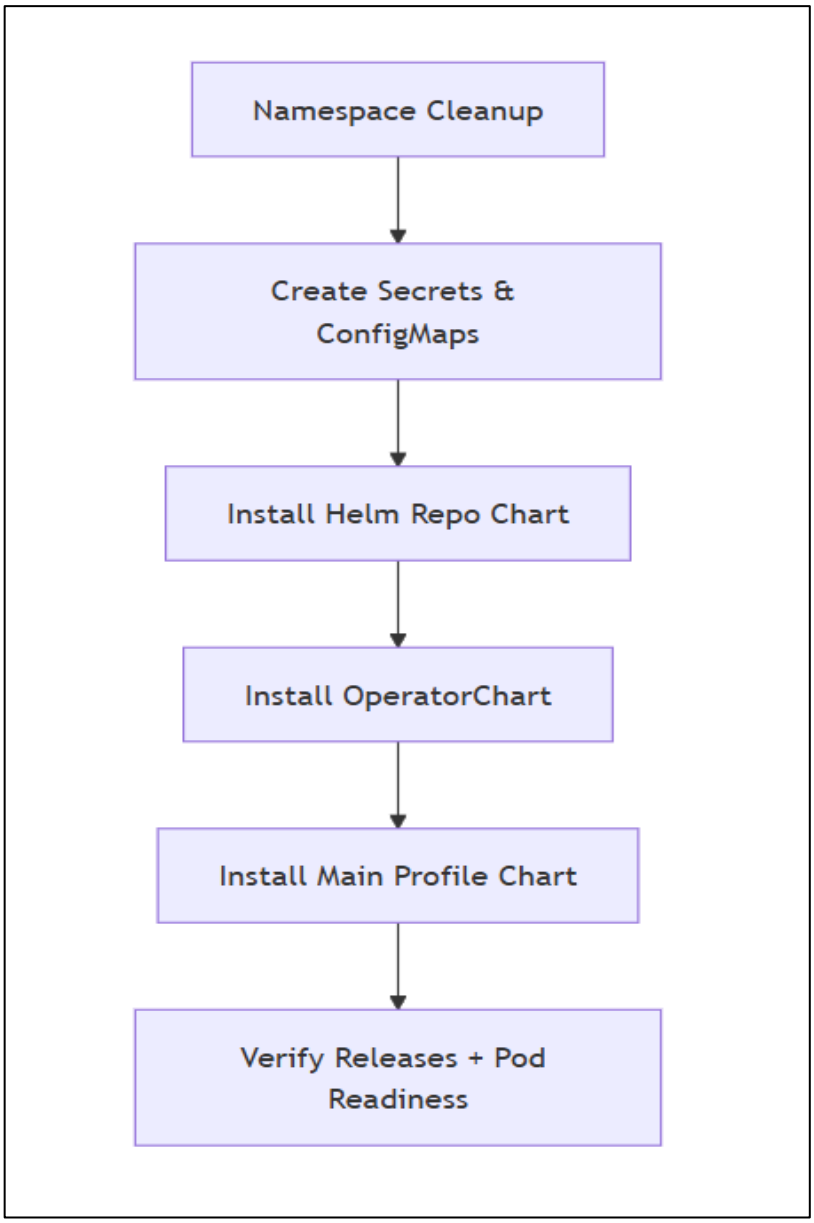

Figure 3.2: K8s Based Deployment

4.5 Post-Deployment Health Checks

A deployment trigger and completion message were treated only as the beginning of validation, not the final success condition. Therefore, post-deployment health verification was performed as a mandatory quality gate before test execution. This aligns with deployment guidance sections that separate “deployment status” from “operational readiness.”

The purpose of this phase was to ensure that all required components were not only deployed but also healthy, stable, and ready for functional/regression test activities.

Post-Check Activities Performed

The following checks were executed after each deployment cycle:

1. **Deployment and release status validation**

   Confirmed that deployment workflow and release objects reached expected completion state.

2. **Pod readiness and restart analysis**

   Verified that mandatory pods transitioned to ready condition and did not show abnormal restart patterns.

3. **Service-level validation**

   Confirmed that required services and endpoints were available and aligned with expected deployment output.

4. **Event and log review**

   Reviewed cluster events and logs to detect hidden or delayed failures that may not appear in high-level status.

CHAPTER 5: WORK DONE – TEST AUTOMATION

5.1 Overview of Automation Work

Test automation was one of the core contributions during the internship. The objective was to reduce repetitive manual testing effort and make regression validation more consistent across deployment cycles. Automation work was aligned with product system testing activities and focused on scenarios that were frequently repeated after deployment, restart, and configuration updates.

The automation stack mainly included:

- **Robot Framework** for regression test execution.
- **Python scripting** for operational and deployment-support tasks..

The automation scope covered both test validation and utility scripting. Regression checks were converted into reusable automated flows, while Python scripts were used to automate environment preparation tasks such as Kubernetes resource creation and deployment command support.

A key outcome of this work was the creation and execution of **6 automated regression scenarios**, which improved repeatability and reduced dependency on purely manual validation.

5.2 Test Cases

Robot Framework was used as the main test automation framework for executing keyword-driven regression scenarios. Python scripts were integrated where command-level and environment-level operations were required.

5.2.1 Automation Coverage

The automated regression scope included six internal scenarios covering:

1. Access and prerequisite validation,
2. Runtime restart behavior checks,
3. Deployment-level recovery validation,
4. Platform stability verification under controlled operations,
5. Configuration-dependent validation flows,
6. Post-action health/state checks.

5.2.2 Execution Approach

The automation execution pattern followed this structure:

- pre-check and environment readiness validation,
- trigger of operation (for example restart/deployment step),
- status and output verification,
- pass/fail logging with traceable output.

Robot suites were used to orchestrate flow, while Python handled lower-level utility and command tasks where needed.

5.2.3 Python Script Contributions

Along with Robot-based regression, Python scripting was used for operational automation:

- **Secret and ConfigMap creation script**

  Implemented with validation and version-aware handling to avoid configuration mismatches.

- **Operator-based Helm command automation**

  Implemented command-driven deployment support in script form for repeatable execution.

- **Support utilities for automation flow**

  Included file/path handling, remote command execution, and command output parsing.

Flowchart Test Cases :

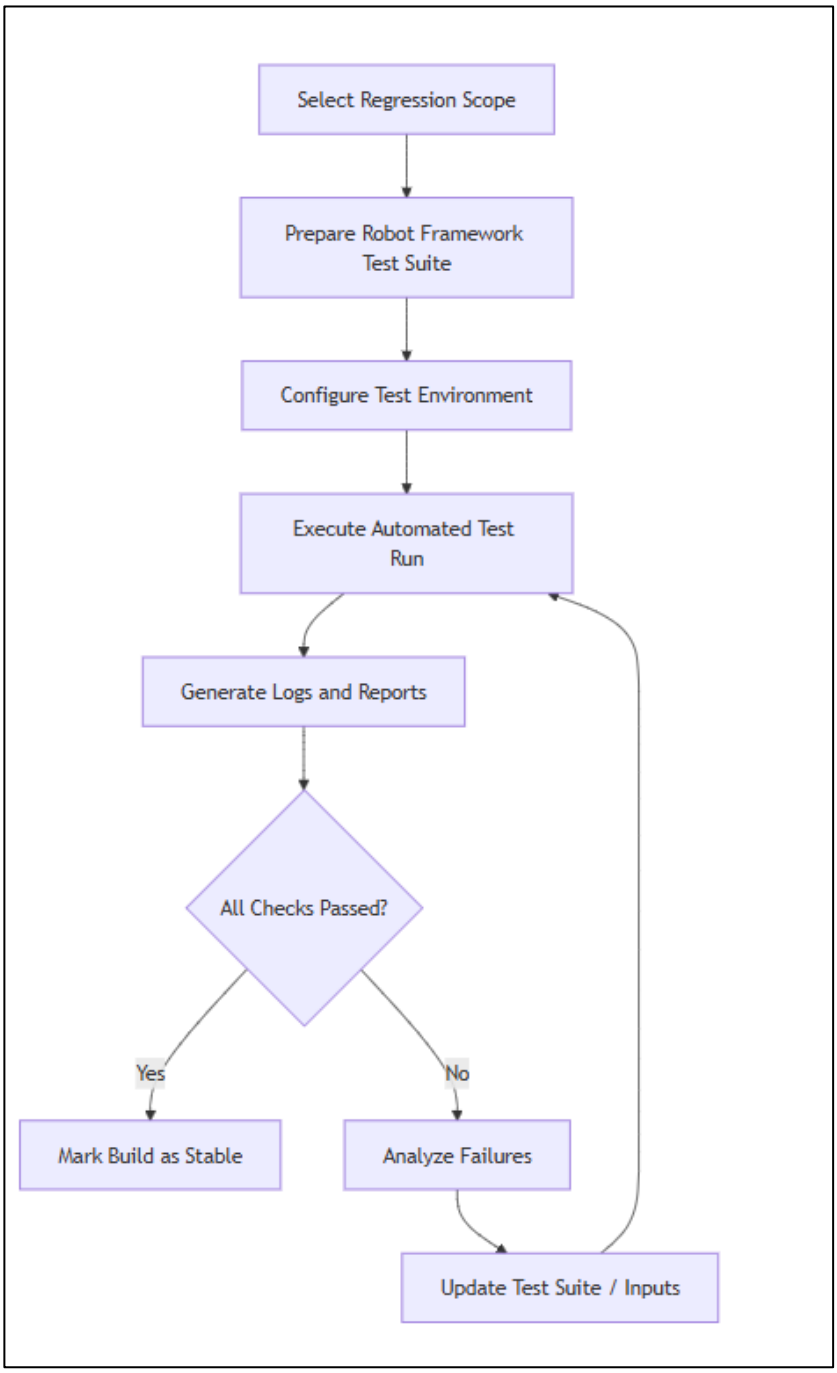

Figure 4.1:Test Cases FlowChart

5.3 Benefits of Automation

The implemented automation provided measurable operational benefits in the workflow.

1. Efficiency Benefits

- Reduced repeated manual effort in regression cycles.
- Faster execution after deployment or configuration changes.
- Quicker verification turnaround during iterative testing.

2. Quality and Consistency Benefits

- Standardized execution steps reduced human variation.
- Consistent validation logic across multiple runs.
- Better reliability in pass/fail decision flow.

3. Troubleshooting Benefits

- Easier issue reproduction using repeatable scripts.
- Improved root-cause analysis through structured output/log capture.
- Faster identification of environment/configuration deviations.

4. Process Benefits

- Better alignment with CI/CD-style repeatable testing practices.
- Reusable automation assets for future regression cycles.
- Stronger foundation for scaling automation in advanced validation stages.

CHAPTER 6: SYSTEM TESTING

Testing is the process of verifying that a system behaves as expected under defined conditions. In telecom product environments, testing ensures that deployment, configuration, and runtime behavior are stable before moving to higher-level validation or production-like scenarios.

In this context, testing was used to:

- confirm that deployed components are operational,
- detect issues early in deployment or configuration,
- verify repeatable behavior across test cycles,
- reduce risk in later upgrade/load activities.

6.1 Testing Approach

The testing approach adopted during the project followed a practical and phased model aligned with cloud-native deployment workflows. In this environment, testing was not treated as a single final activity; instead, it was integrated across all stages of deployment and stabilization. The objective was to ensure that each deployment cycle was technically correct, operationally stable, and ready for subsequent validation activities.

1. Test Objectives

The testing work was guided by the following objectives:

- **Validate system stability after deployment:**

  Confirm that the deployed components reach a healthy and steady state after installation and configuration.

- **Verify repeatability across regression cycles:**

  Ensure that expected behavior remains consistent when tests are re-executed after deployment changes, configuration updates, or recovery actions.

- **Detect and isolate failures at multiple layers:**

  Identify issues at configuration level, deployment execution level, and runtime level to reduce propagation of faults into later stages.

- **Support reliable release readiness:**

Build confidence that each cycle meets baseline quality before moving to deeper testing or next deployment iteration.

2. Testing Stages Followed

The testing process was executed in well-defined stages:

- **Pre-check stage:**

  Verified prerequisite readiness, including artifact availability, configuration completeness, namespace dependencies, and environment accessibility.

- **Deployment stage:**

  Triggered deployment and continuously tracked lifecycle status, chart/release behavior, and execution events to identify early failures.

- **Post-deployment stage:**

  Verified pod readiness transitions, service-level health, and successful application of deployment configuration.

- **Regression stage:**

  Executed repeatable automated validation cycles to confirm that previously stable behavior remained intact after changes.

- **Issue-handling stage:**

  Collected runtime evidence (events, status outputs, logs), analyzed probable root causes, applied corrective actions, and revalidated closure.

3. Practical Characteristics of the Approach

- Iterative execution across multiple deployment cycles
- Strong dependency on readiness and health verification, not only deployment completion
- Failure-driven revalidation model (fix -> rerun -> confirm)
- Emphasis on consistency and reproducibility over one-time pass results

6.2 Deployment Validation Testing

Deployment validation testing was performed to verify that the product was not only installed successfully, but also operationally ready for system-level activities. In cloud-native environments, an installation marked as "completed" does not always mean the runtime state is functionally stable. Therefore, deployment validation acted as a mandatory quality gate between deployment execution and further testing.

1. Purpose

Deployment validation was carried out to:

- **Confirm successful bring-up of deployed components:**

  Ensure required components are initialized and transitioned to expected runtime states.

- **Verify correctness of artifacts and runtime configuration:**

  Check whether deployment artifacts, values, and environment resources are applied as intended.

- **Validate readiness and health after each deployment trigger:**

  Ensure workloads are ready for continued testing and not in partial/unstable states.

- **Prevent downstream instability in regression cycles:**

  Catch deployment-level issues early before they affect test consistency.

2. Validation Activities Performed

The following activities were executed after deployment:

- Checked lifecycle/deployment status for successful completion.
- Verified pod state progression and readiness transitions.
- Confirmed service availability in the target namespace/environment.
- Validated that required configuration resources were properly consumed.
- Performed runtime health checks and sanity verification steps.
- Monitored events and logs for hidden or delayed failures.
- Re-validated system state after corrective actions when issues were detected.

3. Outcomes of Deployment Validation

- Improved confidence in deployment quality before regression execution.

- Reduced false-positive success scenarios (where deployment is complete but unstable).
- Faster identification of configuration and environment-level issues.
- Better continuity between deployment and testing phases.

6.3 Regression Testing

Regression testing was conducted to ensure that validated behavior remained stable after updates, reconfiguration, and repeated deployment cycles. Because the environment was iterative and cloud-native, regression was essential to confirm that fixes and changes did not unintentionally break previously stable functionality.

1. Purpose

The regression process was designed to:

- **Protect existing stability after changes:**

  Confirm that updates do not introduce unintended functional or operational regressions.
- **Maintain consistency across repeated cycles:**

  Ensure expected behavior remains reproducible from one cycle to the next.
- **Reduce dependency on manual re-verification:**

  Improve testing speed and consistency through automation-driven execution.
- **Increase release confidence:**

  Provide measurable validation before proceeding to advanced testing or future scope tasks.

2. Scope

- A set of **6 regression scenarios** was automated and executed.
- Scenario names and internal details are intentionally not disclosed for confidentiality.
- The scope focused on operational reliability, recovery behavior, and stability validation.

3. Execution Method

Regression testing followed a repeatable execution pattern:

- Scenarios were executed through an automation framework.
- Runs were repeated across deployment cycles to measure consistency.

- Outputs were compared with expected validation criteria.
- Failures were analyzed, corrected, and re-executed to confirm proper closure.
- Results were reviewed as part of the cycle-level quality assessment.

4. Observed Benefits

- Faster validation than manual repeated checking
- More consistent execution across cycles
- Improved reproducibility of failures for troubleshooting

CHAPTER 7: RESULTS AND OBSERVATIONS

This chapter summarizes the measurable outcomes and practical observations from the internship work across three major areas: product deployment, test automation, and technical training. The results reflect progress from initial setup stages to stable execution and repeatable validation practices in a cloud-native telecom testbed.

7.1 Deployment Results

The deployment activities were completed through multiple iterations, and each cycle improved process stability and execution confidence.

Key Results

- Product deployment was successfully executed in the test environment using the defined lifecycle workflow.
- Required deployment artifacts (images/charts) were onboarded and consumed through registry-based deployment.
- One-time and recurring prerequisites (security/configuration resources) were applied correctly for deployment readiness.
- Deployment status tracking and post-deployment verification became more structured over time.
- Re-deployment after configuration changes was handled with better control and reduced delay.

Observations

- Deployment success depended heavily on correct artifact names, values/configuration consistency, and sequence adherence.
- Validating prerequisites before trigger significantly reduced deployment failures.
- “Deployment completed” and “deployment stabilized” are different stages; health verification is essential before test execution.
- Iterative cycles helped build a repeatable deployment checklist for future use.

7.2 Automation Results

Automation work focused on reducing manual effort in repeatable validation activities and improving consistency of regression execution.

Key Results

- **6 regression scenarios** were automated using the selected automation framework.
- Repeatable validation flow was established for critical runtime checks.
- Automation reduced manual execution time and improved consistency across test runs.
- Scripted execution improved failure reproducibility and debugging speed.
- Automation support tasks for deployment operations improved operational efficiency.

Observations

- Automation is most effective when preconditions and environment checks are explicitly included.
- Structured test execution produced more reliable pass/fail outcomes than manual-only runs.
- Standardized automated checks improved confidence after deployment or restart events.
- Automation assets can be scaled further for wider regression coverage in future phases.

7.3 Training Outcomes

The training program provided the technical base required for practical deployment and system testing work.

Key Results

- Strong understanding of telecom and IMS fundamentals relevant to product system testing.
- Improved conceptual and operational clarity on cloud-native deployment (Kubernetes/Helm model).

- Better understanding of lifecycle-driven deployment, monitoring, and troubleshooting workflow.
- Exposure to operations context, cross-team workflow, and issue-handling discipline.
- Improved technical documentation quality and reporting structure.

Observations

- Training topics directly aligned with day-to-day deployment and testing activities.
- Conceptual knowledge from IMS and Kubernetes sessions helped resolve practical environment issues faster.
- Combined training + hands-on execution improved both technical confidence and execution quality.
- The internship created a solid foundation for advanced tasks such as broader validation, troubleshooting depth, and load-oriented testing in future work.

CHAPTER 8: CONCLUSION

8.1 Current State of the Work

This internship work established a strong foundation in cloud-native telecom product deployment and system testing. The key outcomes include:

- Successful participation in multiple deployment cycles in the test environment.
- Practical understanding of registry onboarding, lifecycle-based deployment, and post-deployment validation.
- Exposure to Kubernetes and Helm-based operational workflows.
- Automation of selected regression activities using structured test execution approaches.
- Improved capability in troubleshooting deployment and configuration-related issues.
- Better understanding of IMS-related product behavior in a cloud-native setup.

At the current stage, the deployment and validation base is stable enough to move toward deeper **application-side testing** .

8.2 Challenges Faced

During the internship, several practical challenges were encountered:

- Dependency on correct image/chart onboarding before deployment trigger.
- Configuration mismatches between deployment values and registry/runtime expectations.
- Runtime cases where pods were running but not fully ready.
- Repetitive manual validation effort during regression cycles.
- Multi-layer troubleshooting requirement across deployment tool, cluster, and application logs.

These challenges were resolved through iterative validation, configuration correction, improved pre-checks, and better deployment discipline. The process improved with each cycle, leading to more predictable and repeatable outcomes.

8.3 Future Work

The next phase of work is focused on application-side validation. The planned goals are:

1. **Install CNF TB**

   Complete end-to-end setup of the CNF testbed environment for application-side testing.

2. **Successful registration and call between two UEs**

   Configure required parameters and achieve successful user registration and call establishment between two user endpoints.

3. **Troubleshooting on failures**

   Build systematic troubleshooting capability for registration, signaling, configuration, and runtime failures during application-side testing.

4. **Bring up basic load for ISSU testing**

   Prepare and execute basic traffic/load conditions required for ISSU (In-Service Software Upgrade) test preparation and validation.

BIBLIOGRAPHY

1. M. Poikselka, G. Mayer, H. Khartabil, and A. Niemi, *The IMS: IP Multimedia Concepts and Services*, 3rd ed., Wiley, 2009.
2. M. A. Ulema, *Fundamentals of 5G Mobile Networks*, Wiley-IEEE Press, 2021.
3. Nigel Poulton, *The Kubernetes Book*, 2023 edition.
4. Luksa, Marko, *Kubernetes in Action*, Manning Publications, 2018.
5. 3GPP standards documentation (IMS/SIP architecture and procedures), selected technical specifications.
6. Nokia product and training documentation used during internship (internal learning material).

REFERENCES

1. Python Software Foundation. “Python Documentation.” https://docs.python.org/3/ .
2. JetBrains. “PyCharm Documentation.” https://www.jetbrains.com/help/pycharm/ .
3. Robot Framework Foundation. “Robot Framework User Guide.” https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html .
4. Python Package Index (PyPI). “Paramiko.” https://pypi.org/project/paramiko/.
5. Python Docs. “subprocess — Subprocess management.” https://docs.python.org/3/library/subprocess.html.
6. Python Docs. “os — Miscellaneous operating system interfaces.” https://docs.python.org/3/library/os.html.
7. Python Docs. “re — Regular expression operations.” https://docs.python.org/3/library/re.html .
8. PyPI. “wget package.” https://pypi.org/project/wget/ .
9. Kubernetes Documentation. “Kubernetes Concepts.” https://kubernetes.io/docs/concepts/ .
10. Helm Documentation. “Helm Docs.” https://helm.sh/docs/ .
11. diagrams.net (draw.io). “Official Site.” https://www.diagrams.net/ .
12. Wikipedia contributors. “IP Multimedia Subsystem.” https://en.wikipedia.org/wiki/IP_Multimedia_Subsystem .
13. Wikipedia contributors. “Kubernetes.” https://en.wikipedia.org/wiki/Kubernetes .
14. Wikipedia contributors. “Session Initiation Protocol.” https://en.wikipedia.org/wiki/Session_Initiation_Protocol .

APPENDIX

Internship Ongoing Certificate (Nokia)

NOKIA

07th April 2026

To Whomsoever It May Concern

Phone : +91-80-43632100
Fax : +91-80-67618018

Nokia Solutions and Networks India Pvt. Ltd.
(Formally known as Nokia Solutions Networks Pvt. Ltd.)

L5, Manyata Embassy Business Park
Outer Ring Road, Nagavara
Bangalore - 560045, India

Corp Identity No.
U72900DL2006PTC155149

Regd. Address :
Nokia Solutions and Networks India Pvt. Ltd.
1507 Regus Business Center
Eros Corporate Tower Level 15,
Nehru Place, New Delhi–110019,
India

This is to certify that **Ashwarya Bharti, 63011200** (**USN: 24MCA10051)**, final year **M.C.A (Master of Computer Application)** student at Vellore Institute of Technology, Bhopal, has been undergoing Internship program in Nokia Solutions & Networks India Pvt. Ltd., from 18th August 2025 as part of the academic program.

Ashwarya Bharti, is involved in **"Autonomous Network for IMS Voice Core"** project of CFX product.

(Deepak SG)
Engineering Manager
IMS & NextGen Voice Product Test Leader
MI CN IPT Session Management

http://www.nokia.com

© Nokia 2026